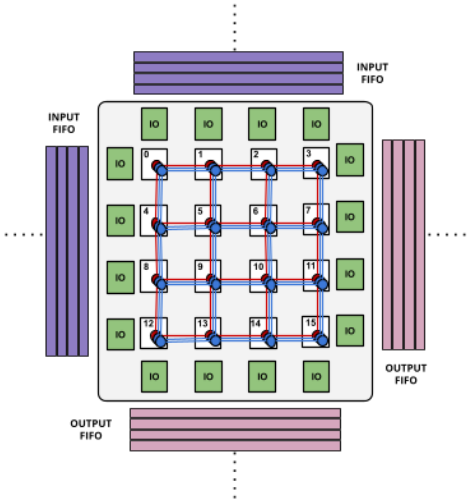
Introduction

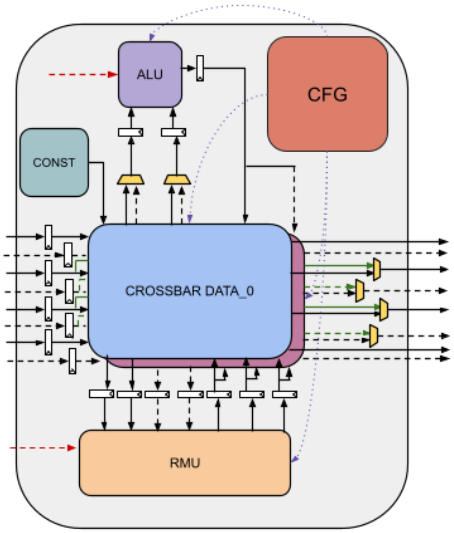
A. Overview

This project implements a dual-mode reconfigurable spatial accelerator for machine-learning workloads. Modern AI models rely heavily on dense tensor operations such as matrix multiplication, linear layers, and convolutions. However, end-to-end inference is not composed only of matrix multiplication. Between these dense layers, models also execute nonlinear and elementwise kernels such as activations, reductions, comparisons, normalization-related arithmetic, and data-format transformations. Conventional systolic arrays are highly efficient for regular multiply-accumulate computation, but they are less naturally suited to these more flexible kernels. This project addresses that gap by allowing the same two-dimensional processing-element fabric to operate in two modes: a systolic mode for regular matrix-style dataflow, and a reconfigurable spatial mode for CGRA-style integer kernels. The purpose of the design is to demonstrate that a single physical fabric can reuse datapath, interconnect, and configuration resources across both dense tensor computation and more general spatial pipelines.

At the chip level, the design consists of a tiled array of integer processing elements connected through configurable routing resources. In systolic mode, the array supports structured nearest-neighbor data movement and regular multiply-accumulate style computation. In reconfigurable mode, the same fabric can be programmed to implement streaming integer kernels such as ReLU, leaky ReLU, elementwise addition and subtraction, comparisons, shifts, simple reductions, and other low-precision operations. This makes the chip a prototype for a more area-efficient accelerator organization. Instead of placing a dedicated matrix engine and a separate vector or scalar unit side by side, the design explores whether one reconfigurable array can provide both deterministic tensor flow and flexible spatial execution.

From a realistic impact perspective, this chip is a class research prototype, but it still involves practical engineering constraints. For public health, safety, and welfare, the design is not safety-critical, but it must still be tested safely using proper ESD handling, voltage limits, and power checks during bring-up. From a global and manufacturability perspective, the chip uses a realistic ASIC flow with area, timing, routing, and physical-design constraints, making it more grounded than a purely simulated architecture. Culturally and socially, the design targets researchers and hardware developers who need flexible accelerators for changing machine-learning workloads, so programmability and usability are important. Environmentally, the design is motivated by reducing unnecessary data movement and improving area efficiency, although fabrication and electronic waste remain concerns for any chip. Economically, the main benefit is hardware sharing: by reusing one fabric for both systolic and reconfigurable execution, the design explores a lower-area alternative to building separate matrix and vector-style accelerators.





B. Related work

Systolic-array accelerators are a natural starting point for this project. The Google TPU demonstrated the effectiveness of using a domain-specific matrix engine for neural-network inference, especially for dense tensor operations such as matrix multiplication. Open-source accelerator generators such as Gemmini similarly focus on configurable systolic arrays and provide infrastructure for evaluating deep-learning hardware designs. These systems show why regular matrix dataflow is attractive: it enables high multiply-accumulate utilization, predictable communication, and efficient local data reuse. However, a purely systolic organization is less flexible for non-matrix kernels that require comparisons, elementwise arithmetic, reductions, or different pipeline structures. This project builds on the efficiency of systolic arrays, but adds a second operating mode so the same physical fabric can also behave as a programmable spatial array.

Other neural-network accelerators, such as Eyeriss, emphasize spatial dataflow, local reuse, and efficient on-chip communication to reduce memory movement. This line of work is closely related because it treats data movement as a central design challenge rather than focusing only on arithmetic throughput. However, many spatial DNN accelerators are still specialized around a particular class of layers, dataflows, or neural-network computations. In contrast, our design focuses on a small integer fabric that can switch between structured systolic execution and more general CGRA-style execution. The key distinction is that the same processing elements and routing fabric are reused for both modes, rather than dedicating separate hardware blocks to each computation style.

Coarse-grained reconfigurable arrays provide another important comparison point. CGRAs use word-level processing elements connected by configurable interconnect, making them more efficient than fine-grained FPGA fabrics for many arithmetic kernels while remaining more programmable than

fixed-function accelerators. Prior CGRA work motivates the use of spatial pipelines for streaming computations, loop kernels, and flexible data movement. However, CGRAs are often not optimized for the highly regular and deterministic communication pattern of systolic matrix multiplication. Our work fills the gap between these two architectural styles by exploring whether a modest chip-scale fabric can support both TPU-like systolic execution and CGRA-like programmable execution without duplicating the main compute array.

The motivation for this project is therefore architectural integration. Prior tensor accelerators provide efficient dense multiply-accumulate throughput, while prior CGRAs provide flexible spatial execution. Real machine-learning workloads need both capabilities: matrix-heavy layers dominate the operation count, but nonlinear, elementwise, and reduction-style kernels still affect latency, energy, and data movement. By sharing hardware between these two execution styles, this chip explores a middle ground: a physically implemented prototype of a reconfigurable tensor array that can demonstrate both matrix and non-matrix kernels on the same silicon fabric.

C. Learning

Given that the majority of team members had not yet taken EE477 and none had prior tapeout experience, an early priority was building familiarity with the ASIC flow from RTL to silicon. The most effective strategy was working through the EE477 lab material ahead of schedule, which provided structured coverage of the flow, key reports to analyze at each stage, and documentation on relevant BSG modules such as bsg_link.

A parallel challenge was that several members were unfamiliar with systolic array architectures, CGRAs, and reconfigurable computing more broadly, including how to design an external interface for communicating with the fabric. To address this, the team consulted a range of resources including the HyCube, Stream-Dataflow, and Google TPU papers, the textbook *Reconfigurable Computing*, and supplementary video material. Particularly valuable was comparing the external interface paradigms used across these works, which informed the architectural decisions made for our own design, such as our ISA and how we set up our external memory.

D. Industry standard

This design uses BSG-Link, sourced from the BaseJump STL library, as the primary high-speed interface between the FPGA host and the chip. BSG-Link is a source-synchronous DDR serial link. Flow control is managed through a credit-based scheme where the receiver returns tokens over a dedicated token clock line, and the transmitter may only send as many flits as it holds credits. This

prevents FIFO overflow across the two domains. A bidirectional channel consists of two independent links — upstream and downstream — each with its own clock domain, FIFO, and credit counter. To the core logic, the link presents a standard valid/yumi handshake interface; toward the pads, it presents serialized DDR signals.

In this implementation, the upstream and downstream are instantiated from BaseJump STL through a wrapper module bsg_link_wrapper.sv, carrying a 32-bit core-facing flit over a 16-bit physical channel via 2:1 serialization. The reset sequencer in chip_top.sv follows the BSG-Link initialization protocol: io_link_reset is asserted, async_token_reset is pulsed for at least one cycle and held for a minimum of four io_clk edges, then deasserted. Token clock and credit return signals are routed to the FPGA host in board_top.sv as required by the protocol.

Communication between the host PC and the FPGA is handled over UART. The FPGA acts as a passthrough, receiving serialized commands and data from the PC via a UART RX interface and forwarding them to the chip over the BSG-Link channel. Return traffic from the chip travels the reverse path, with the FPGA serializing responses back to the PC over UART TX.

Planning

A. Design specification

The goal of this project is to design a dual-mode reconfigurable accelerator capable of executing both systolic array style matrix multiplication and CGRA-style integer kernels. Instructions are provided by the external control logic, data is routed into the fabric, and executed across the fabric before being returned as output vectors through the external interface.

The effectiveness of the design is measured through both systolic array and CGRA metrics. Metrics such as array size, throughput, latency, utilization, area, and power are taken into account. These metrics represent the performance of overall design.

The array size determines the number of processing elements in the fabric. This directly impacts throughput, area, and power, with larger arrays being able to execute more operations in parallel but increasing routing and control complexity and physical design cost.

Throughput measures the amount of computation per cycle during steady state execution. This metric helps measure the processing capacity of our fabric, especially for a regular execution model like the systolic array mode.

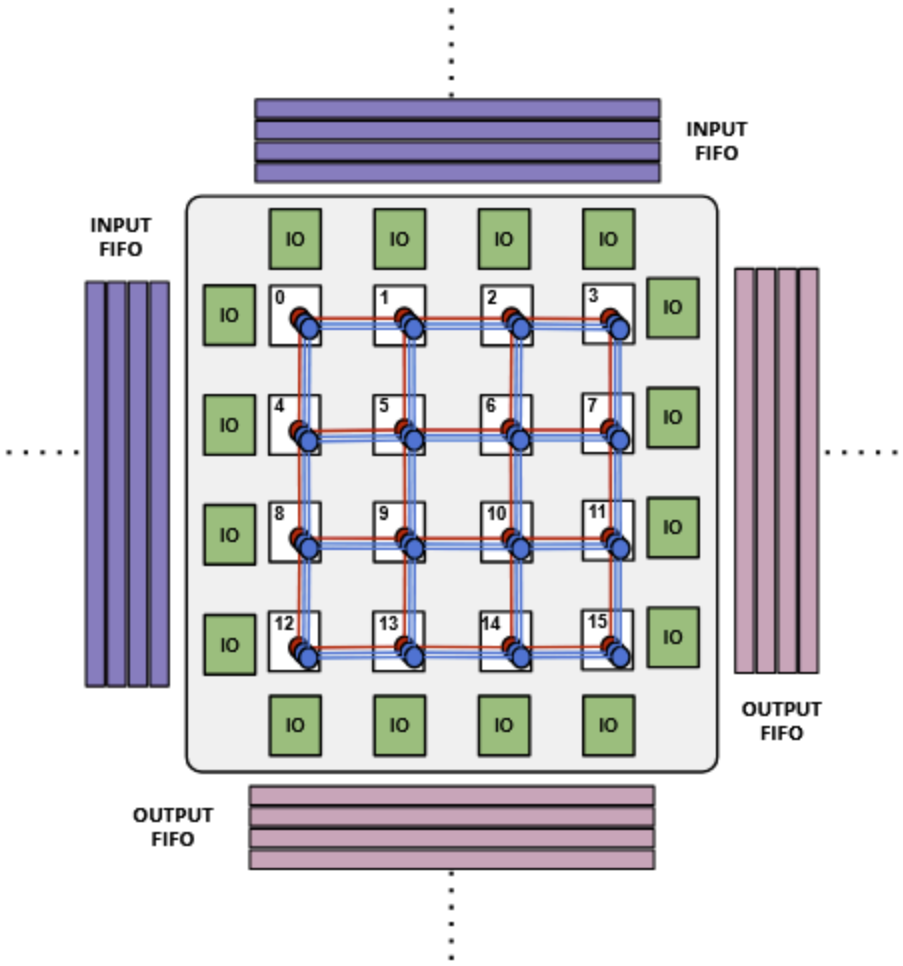
Latency is measured as the number of cycles between when an input is accepted and when the corresponding output is valid. This is useful for understanding the total cost of executing a kernel or operation, especially when configuration overhead is high.

Utilization measures how effectively the available processing elements are used during execution. High utilization demonstrates that the mapping and dataflow is efficiently using the fabric, while low utilization may indicate a bottleneck or insufficient parallelism.

Area is constrained by the maximum chip size of 1.7 mm x 1.7 mm. The design must fit completely within this area budget. Area is evaluated after synthesis and physical implementation, with careful checks to ensure that we do not exceed our allotted area.

Power is evaluated to estimate the energy consumption of running our design. Estimates are taken using the power reports produced by the tools in our flow.

At the top level, our design consists of the core fabric and an external interface. The core fabric is structured as a 4x4 array of processing elements, with 4 I/O modules on the north, south, east, and west sides of the array. The processing elements are designed to support INT8 operations in CGRA mode. In systolic array mode, the data size is lowered to INT4 and we are able to simulate the functionality of a 8x8 systolic array. Connecting the processing elements are two INT8 data planes and one 1-bit control plane. Accumulations are handled using a 16-bit internal register.

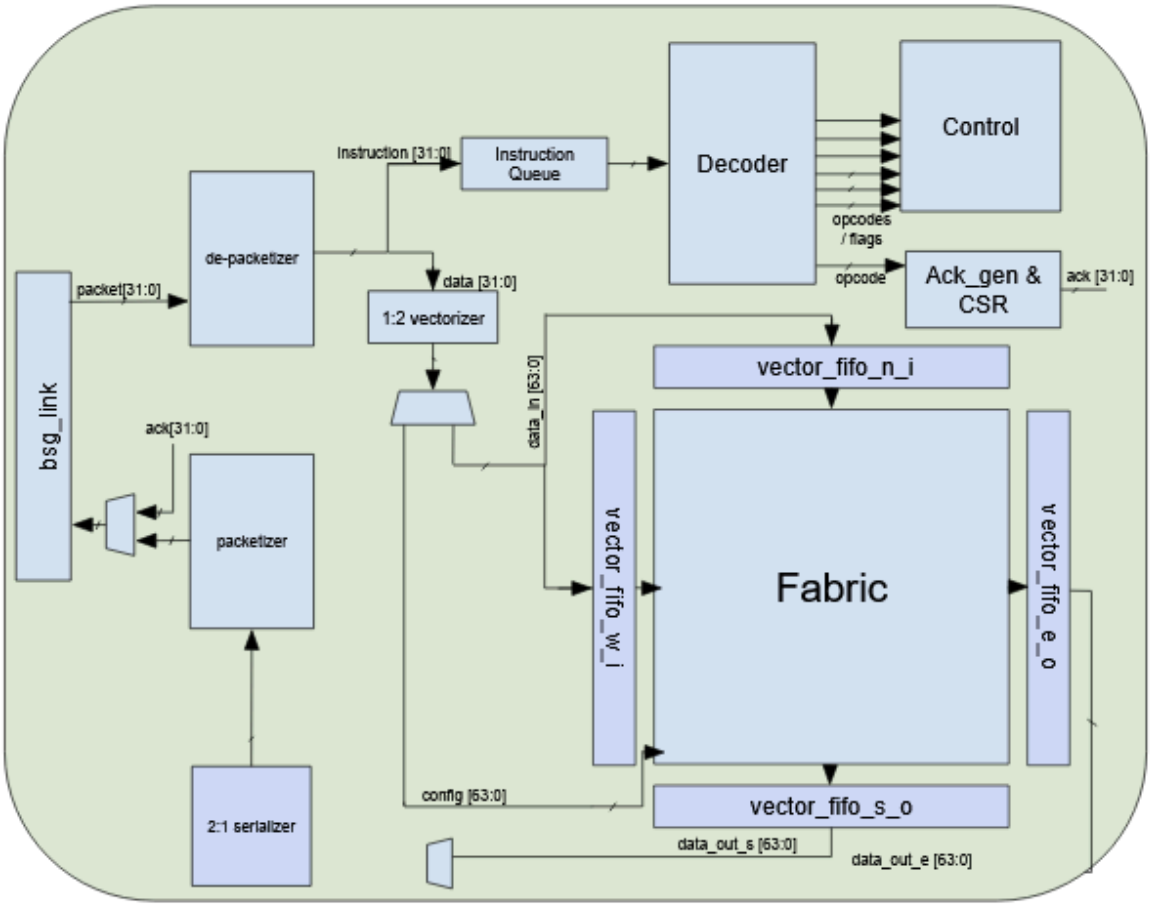


The RTA-V4 processing element includes a Reconfigurable Multiply Unit (RMU) that shares one compact multiplier datapath across both normal CGRA execution and systolic matrix-style execution. Architecturally, the RMU has four 8-bit operand inputs from the two data planes and two 8-bit result outputs, controlled by a 5-bit configuration word. Internally, it uses four shared unsigned 4x4 multiplier lanes. In CGRA mode, those lanes generate the partial products for a signed 8x8 INT8 multiply, then recombine and sign-correct them into a 16-bit result split across `data0` and `data1`. In systolic mode, the same lanes treat the two 8-bit data planes as four packed signed INT4 lanes, compute a dot4 operation each cycle, and accumulate the sum into a local 16-bit accumulator. Separate clear, compute, and drain operations allow the array to reset accumulators, stream multiply-accumulate work across cycles, and later drain the accumulated result back into the PE data crossbars. This lets RTA-V4 support both scalar CGRA multiply kernels and systolic GEMM-style dataflow without duplicating multiplier hardware.

Surrounding our core fabric is four FIFOs, with input FIFOs being on the north and west sides and output FIFOs on the east and south side. These FIFOs are designed to hold input packets and allow us to collect and stream in inputs without the need for an on-chip memory.

The external datapath is designed to handle incoming instructions and packets and send output packets back over BSG-Link modules. Inputs are de-packetized before being sent to either the instruction queue or vectorizer. Instructions are sent to the instruction queue, where they are then decoded and the appropriate control signals are then asserted. Data packets are collected in the vectorizer until they reach the correct length and the resulting vector is sent to either of the input FIFOs, depending on the current routing configuration. Outputs are read from the output FIFOs before being serialized and sent back over the BSG-Link module

Inputs are given as packets split into instruction and data packets. Data packets represent raw data that is intended to be fed to the core fabric. Instructions are specified according to the ISA and are processed separately from data. Outputs are either Ack packets that are generated from the instruction handling logic or they are output vectors from the core fabric.



The chip is intended to be interfaced by an FPGA to send and receive data. The FPGA contains a paired BSG-Link module to support all data transfer between the chip and FPGA. Packets are provided either from the FPGA side onboard memory or using a UART connection between the FPGA and a host PC. Received outputs from the chip are streamed to the host PC over UART. The FPGA is also responsible for providing clock and reset signals to the chip.

The design targets a maximum physical area of 1.7 mm x 1.7mm and a clock frequency of 50MHz. These constraints guide the size of the fabric, the depth of buffering and the overall complexity of our design.

B. Collaboration

- Shwet: Core CGRA fabric prototyping + Bitstream generation
- Eric: RTL and testbenches + simulation for external interface modules, ensure that it is compatible with fabric. Assist in FPGA emulation and physical design.
- Fergus: FPGA RTL and simulation/emulation, testing both the FPGA side and chip side in simulation. Assisted in physical design and architecture design.
- Sheehan: Physical Design, Assisted in RTL design along with simulation for external interface modules
- Rampranav: Carried out tasks for Physical Design and ensuring chip constraints are met. Assisted in Verification & RTL Design alongside Physical Design.
- Aayush: FIFO modules + LEC

Although responsibilities are divided, all members will assist each other in RTL development (w/ DORA), verification efforts, timing closure, and pushing through the HAMMER physical design flow. Slack will be the primary form of communication along with in-person meetings 3 times a week. RTL and any related code will be shared using a GitHub repository. All work done on the Linux Lab machines will be done in a shared directory so that we can easily review any generated reports and contribute to any tasks without needing to restart the overall flow.

C. Timeline & Milestones

Week	Planned	Actual
1		
2	Finish core fabric RTL, finalize external architecture	
3		

4	RTL Freeze, fully verify each individual module	Finish core fabric RTL, finalize external architecture
5	Get full run through flow until PAR	
6		RTL Freeze, fully verify each individual module
7	Finish end-to-end testing for sim-rtl	Get full run through flow until PAR
8	Fully verified design flashed and tested on FPGA, sim-syn and sim-par	Finish end-to-end testing for sim-rtl
9		Fully verified design flashed and tested on FPGA, sim-syn and sim-par
10	Prepare for submission; final signoff complete	Prepare for submission; final signoff complete

-

Design

A. RTL

External Designs and Packages

Two external packages are incorporated into this design. BaseJump STL provides bsg_link_ddr_upstream, bsg_link_ddr_downstream, and associated credit-counter primitives, used verbatim. The CGRA PE array is generated using DORA, a programming-first CGRA design framework that uses a Python architecture description and emits synthesizable SystemVerilog. The instantiated array (rta_v4_array in cgra_top.sv) is entirely DORA-generated output; all surrounding RTL was authored by this team.

Notable RTL Features

RTL Generation. The CGRA PE array is produced by DORA from a Python architecture description rather than hand-authored RTL.

Parameterization. All modules are fully parameterized. Most notable is the FIFO_ELS parameter to set the depth of the FIFOs in our design. This makes for a simple way to reduce / increase area if necessary. The CGRA configuration bit count flows from a single top-level define (RTA_V4_CONFIG_BITS = 2084) through CONFIG_BITS_p into the configuration serializer and stall controller, ensuring consistency across the hierarchy without hardcoded constants.

FSMs. Four FSMs implement the host communication datapath. The depacketizer FSM (states: S_FETCH, S_DATA_WR) parses the incoming 32-bit word stream from BSG-Link: in S_FETCH it decodes the opcode and either enqueues an instruction or latches the payload word count before transitioning to S_DATA_WR, where payload words are streamed to the datapath one word per cycle until the count reaches zero. The data_concat FSM (states: S_W0, S_W1) accumulates two consecutive 32-bit words into a single 64-bit vector before presenting it to the fabric input FIFOs. The packetizer FSM (states: S_IDLE, S_HDR, S_DATA) frames read responses by generating a header word (opcode 0xE plus length) followed by N payload words drained from the output FIFO and forwarded to the host over BSG-Link. Finally, the uart_prog_loader FSM (states: S_IDLE, S_RX, S_PRESENT, S_DONE) assembles incoming UART bytes into 32-bit commands using a 5-byte little-endian record protocol.

Instruction Pipeline. The decoder contains a single pipeline register between the instruction FIFO and the control unit. An instruction is fetched from the FIFO in cycle N and its decoded fields are presented to the control unit in cycle N+1. The pipeline stalls — holding the register and suppressing new fetches — whenever the ACK generator has a pending undelivered acknowledgment or a read transaction is in progress.

Handshaking. All inter-module interfaces use one of two latency-insensitive handshake protocols. The BSG-Link core-facing interface uses valid/yumi, where a transfer fires when both valid and yumi are asserted and yumi is driven by the consumer. Internal paths — including depacketizer-to-FIFO, data_concat to ring, and the packetizer — use valid/ready, where a transfer fires when both signals are asserted. Backpressure propagates from downstream to upstream: the depacketizer stalls on instruction words when the FIFO is full and on data words when the data_concat FSM is not ready. The packetizer de-asserts core_valid_o mid-burst if the output FIFO empties, stalling the link until data is available.

Sticky Control Registers. Several control signals are implemented as registered sticky bits rather than single-cycle pulse outputs, depending on their specific use. bus_route_sel in control.sv is set by a WRITE_CONFIG instruction and cleared by WRITE_DATA, steering subsequent data words to either the fabric input FIFOs or the configuration path without requiring the host to re-issue the routing command with every word. Similarly, write_sel_r and read_sel_r in external_top.sv latch the direction from the decoded instruction and hold it for the duration of the transaction. However, control signals such as run, mem_rst, fabric_rst were asserted as single cycle pulses, as there was no need for them to be held high for more than one cycle.

In-Order ACK Generation. ack_gen.sv produces one acknowledgment packet per decoded instruction. A one-deep register holds the pending ACK, and ack_ready_for_instr_o is de-asserted while an acknowledgment awaits acceptance, stalling the decoder pipeline until the downstream off-ramp accepts the word. This means that the chip's control logic stalls indefinitely until the ack is accepted.

Each ACK carries a CSR status snapshot and the current program counter, providing the host with visibility into chip state after every instruction.

External Interface. Communication between the host PC and the FPGA is conducted over a 115,200-baud UART. The FPGA acts as a passthrough: the UART RX path feeds the uart_prog_loader FSM, which assembles incoming bytes into 32-bit commands that are forwarded to the chip over BSG-Link. Return traffic follows the reverse path, with the UART TX path serializing 32-bit response words from BSG-Link back to the host. The PROG_SOURCE parameter on board_top selects between UART mode for runtime streaming and MEM mode for compile-time $readmemh-based simulation and static hardware loading.

DFT Scan Chain. A scan chain, enabled via an `ifdef DFT_EN guard, is exposed through three dedicated pads: PAD[11] (scan_en), PAD[46] (scan_in), and PAD[47] (scan_out).

Pad Ring. chip_top.sv instantiates a 48-pad TSMC 180nm pad ring consisting of bidirectional IO pads plus corner power and ground cells on each side.

B. Verification

Verification was done using end-to-end simulation, post-synthesis simulation, post-PAR simulation, scan-chain testing, and FPGA prototyping. The goal was to test the chip through its real external interfaces by sending instructions and data into the design, running kernels on the fabric, and checking the returned output vectors.

The main chip-top testbench we built was tb_chiptop_gemm. It instantiates the full chip_top, drives the package-level pads, models the FPGA-side BSG-Link endpoint, programs the fabric with generated bitstreams, streams input data into the chip, and reads outputs back from the host-visible interface. The primary workload was GEMM. The testbench computes a software golden model for INT4 dot products, sends matrix inputs through the west and north sides of the fabric, drains the output columns, and compares the returned 16-bit accumulator values against expected results. We tested directed and randomized cases including zero inputs, signed values, max/min values, lane-sentinel patterns, late-pulse cases, and multiple K sizes.

We also wrote smaller chip-top testbenches for bringup. chip_top_tb.sv tested pad connectivity, BSG-Link communication, SPI control, reset sequencing, and status readback. chip_top_tb_scan.sv tested the DFT scan chain by driving the JTAG TAP, shifting in a known pattern, and checking the scan output.

After RTL bring up, we ran sim-syn and sim-par through Hammer. sim-syn verifies the synthesized netlist with SDF annotation, while sim-par verifies the post-place-and-route netlist with post-PAR SDF annotation. Both used the same tb_chiptop_gemm workload, and both completed all GEMM cases with tb_chiptop_gemm PASS.

FPGA prototyping was conducted to validate the chip design before final tapeout. The prototype setup allowed us to test the chip interface, stream ISA instructions into the design, observe output vectors, and verify system behavior under realistic communication and control conditions. The FPGA side was set up to act as the test driver for the chip, with both the chip and the FPGA side modules being synthesized onto the board and connected using GPIO pins, simulating the post-silicon testing environment. Instructions were passed from the host PC to the FPGA over UART, which were then sent to the chip. After execution, the chip produces result vectors, which are sent back to the FPGA, and transmitted to the host PC for collection and verification. We tested simple kernels such as matrix multiplication and ReLU, simulating a realistic workload for our chip. These were evaluated for correctness and helped expose issues such as reset sequencing.

C. Physical Design

The design was implemented using a flat physical design flow. We did not use a hierarchical flow because the fabric was not large enough to justify creating separate hard macros and integrating them later. Keeping the design flat made iteration simpler and allowed the tools to optimize across the full chip.

A manual floorplan was used to organize the major blocks. The CGRA fabric was placed near the center of the core, with the north, south, east, and west FIFOs placed around it to match the expected data movement through the array. The external interface logic was placed near the chip I/O side to reduce unnecessary routing between the pads, BSG-Link wrapper, instruction path, and output path.

Design-for-test was included through scan-chain insertion. Scan enable, scan input, and scan output were routed through existing chip pads, so no extra top-level ports were required. Some blocks were excluded from scan where needed, such as clock-domain crossing logic and interface logic that could not be cleanly controlled by the scan clock.

After place and route, the design was checked with DRC, LVS, antenna, and timing reports. Three remaining metal-related DRC issues were fixed manually by adding dummy Metal 6 blocks in the affected locations. These fixes did not change the RTL functionality, but allowed the final layout to pass the physical signoff checks.

D. Optimization

External Optimizations:

- No memory, FIFO only: The design avoids large on-chip SRAMs and instead uses input/output FIFOs around the fabric. This greatly reduced area and simplified the external datapath and ISA (no longer need memory operations), while still allowing the chip to stream data into and out of the array.
- Minimal ISA / control overhead: Knowing our area is limited, we aimed to minimize the control logic overhead. As a result, we implemented 5 instructions, a minimal amount to write/read data, reset components, and start execution.

Core Fabric Optimizations.

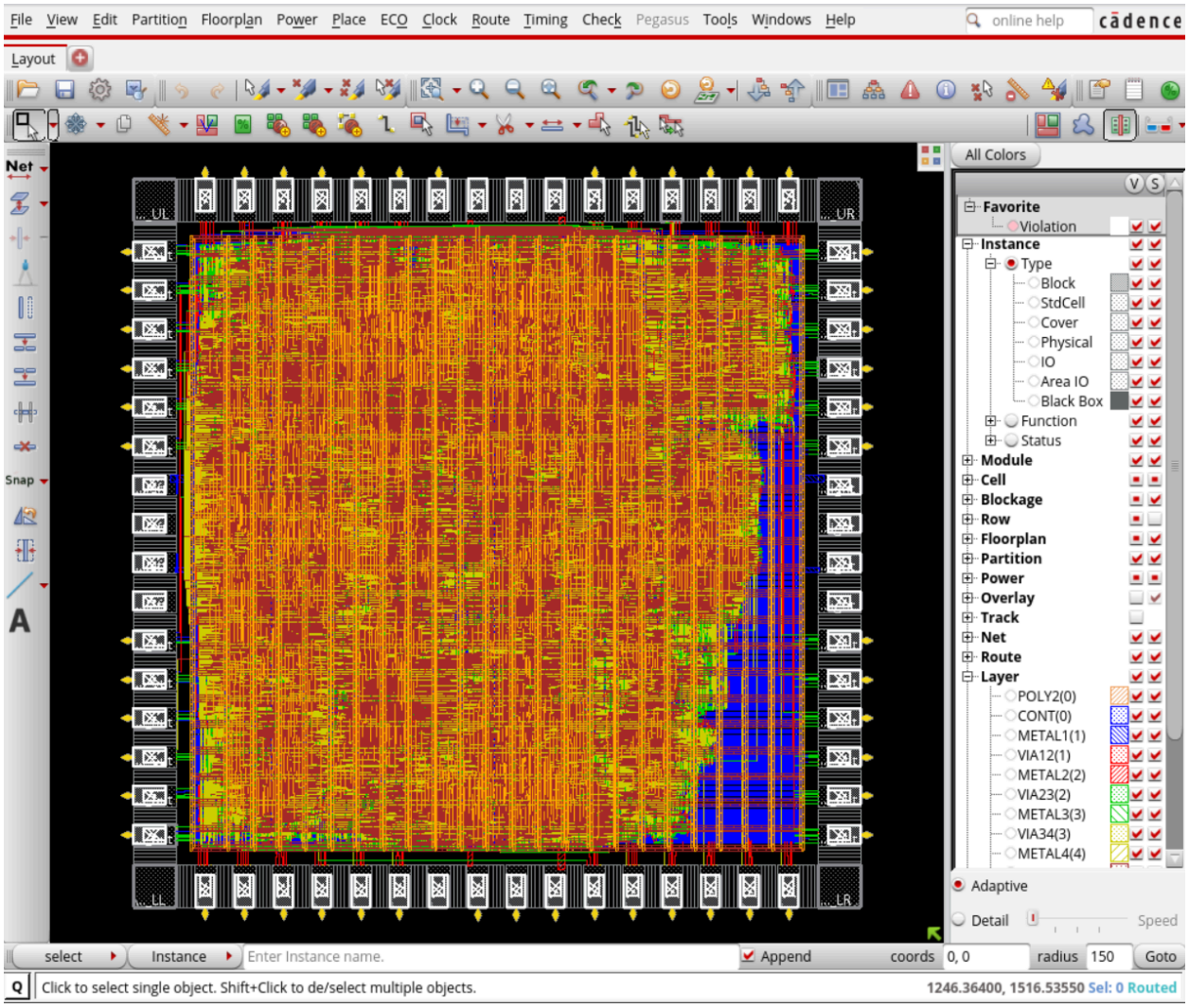
- Outer Product GEMM to DOT4-style ACC. Outer product style GEMM dataflow required the drain phase to pump out 4 INT16 accumulators per PE, i.e, 64 accumulators in total. We moved to a DOT4-style GEMM dataflow, which involves a single INT16 accumulator per RMU per PE while maintaining the same amount of MULs. Thus, this reduced the number of accumulators that needed to be drained and held on the fabric.
- Dual 8-bit NoCs. We decided to split our 16-bit single into 2x 8-bit dual NoCs at the cost of higher configuration overhead to improve routability for the pipelined mode of our fabric.
- INT8 -> INT4: Systolic mode uses packed INT4 values instead of full INT8 values. This allowed each PE to perform multiple smaller multiplies per cycle using the shared RMU hardware, improving compute density and reducing multiplier cost.
- RMU: The RMU shares one compact multiplier datapath across both CGRA and systolic modes instead of using separate multiplier hardware for each mode. This reduced area while still supporting signed INT8 multiplication in CGRA mode and packed INT4 DOT4 accumulation in systolic mode.

Physical Design Optimizations

- Floorplanning: The fabric was placed near the center of the core, with FIFOs positioned around it according to the north, south, east, and west data movement directions. External-interface logic was placed closer to the I/O side, reducing unnecessary wirelength and helping with timing and routing.

Evaluation Results

A. Layout Photo



B. Quality Metrics

Quality metrics were collected from the ASIC flow reports. Area was taken from the post-PAR Innovus area report, layout density from the Innovus post-route summary, and setup/hold timing from the post-RCX Tempus STA reports. Timing was checked across the MMMC corners used by the flow: WCCOM setup at 1.62 V and 125 C, BCCOM hold at 1.98 V and -40 C, and NCCOM extra at 1.80 V and 25 C. The main chip supply is 1.8 V, with 3.3 V used for the pad ring.

The final layout die size from the DEF is 1964.88 um x 1964.88 um. The post-PAR standard-cell area reported by Innovus is 2,164,021.971 um^2, and the final post-route placement density is 68.802%. From post-RCX Tempus timing, the minimum setup slack is 0.541 ns, so the design meets the 50 MHz setup target. The

minimum hold slack reported is -0.201 ns, coming from one scan-mode hold path; the functional hold view is clean, but the available report still shows this remaining scan hold violation.

For design-specific metrics, the RTA fabric is a 4x4 array. In CGRA mode, this gives up to 16 PE operations per cycle, depending on the mapped kernel and routing utilization. In systolic INT4 mode, each PE can perform a DOT4-style operation, giving a theoretical peak of 64 INT4 MACs per cycle across the 16 PEs. At the 50 MHz target clock, this corresponds to a theoretical peak of 3.2 GMAC/s for systolic mode, or 6.4 GOPS if counting multiply and add as separate operations.

C. Analyses

Overall, the final results are satisfactory because the design meets the main goals of the project. The chip successfully demonstrates a dual-mode fabric that can operate in both CGRA mode for flexible integer kernels and systolic mode for INT4 matrix-style computation. The design also passed RTL, post-synthesis, post-PAR, and FPGA-prototype testing, showing that the architecture is functional across multiple verification stages.

The physical design results are reasonable for a class research prototype. The final layout size is 1964.88 um x 1964.88 um, with a post-route placement density of 68.802%. The design meets the 50 MHz timing target with a minimum setup slack of 0.541 ns. This suggests that the RTL structure, pipeline placement, and floorplanning were effective. The remaining -0.201 ns hold slack occurs in a scan-mode path, while the functional hold view is clean, so the main improvement would be to fix scan-mode hold timing in a future revision.

The architecture also shows useful performance potential. In CGRA mode, the 4x4 array can perform up to 16 PE operations per cycle. In systolic mode, the fabric can reach a theoretical peak of 64 INT4 MACs per cycle, or 3.2 GMAC/s at 50 MHz. The shared RMU datapath was an important optimization because it allows the same multiplier hardware to support both INT8 CGRA execution and packed INT4 systolic execution.

The main tradeoff is that flexibility comes with overhead. A dedicated systolic array would likely be more efficient for matrix multiplication alone, while a dedicated CGRA could be simpler for flexible kernels. However, this design intentionally explores hardware reuse between both modes. Future improvements could include closing the remaining scan hold violation, adding small on-chip memories for better data reuse, improving the host communication bandwidth beyond UART, and collecting more detailed workload-specific power results.